Git & GitHub

Coders.Eleven – PYTHON ADVANCED

Coders.Eleven PYTHON ADVANCED
Git & GitHub

#1 Allgemein

Git vs. GitHub

Versionskontrollsysteme

Geschichte von Git

Installation

#2 Stages & Files

Stages

Files

Lizenzen

#3 Terminal

Grundlegende Befehle

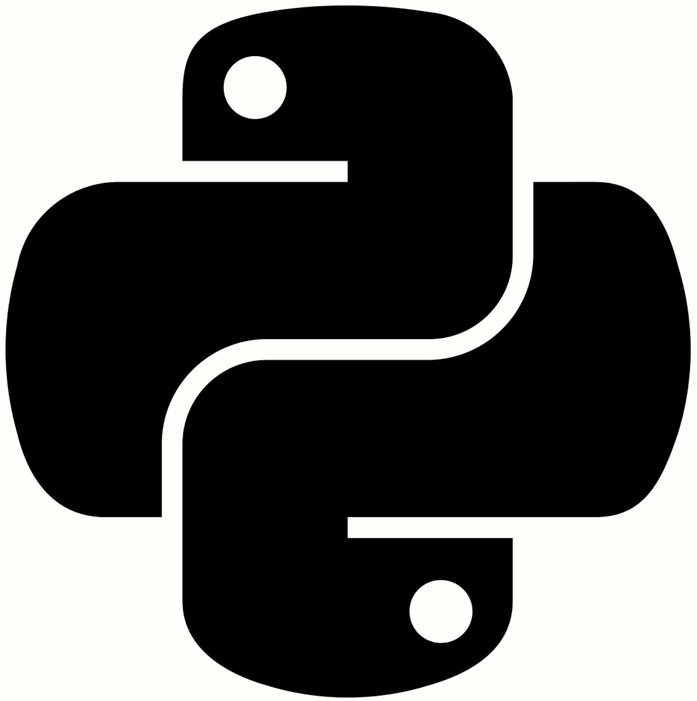
Git Befehle

HTTPS vs. SSH

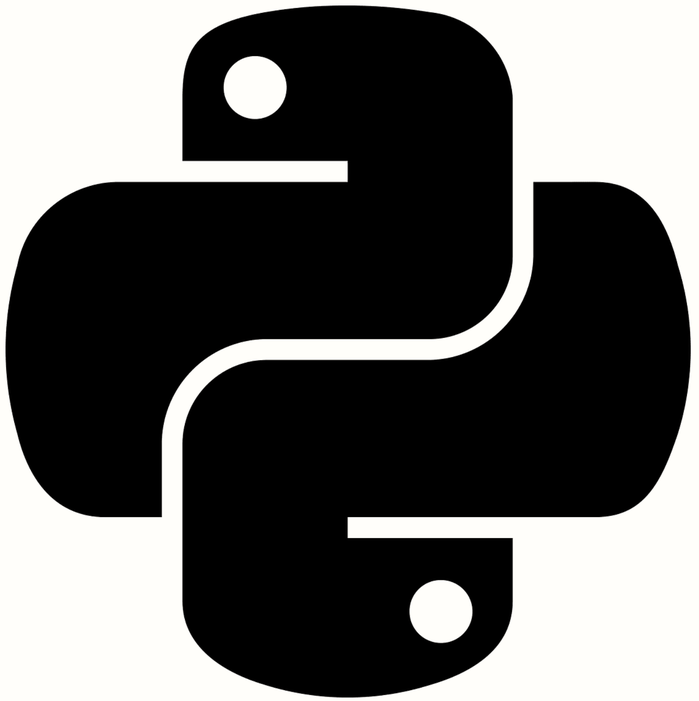
#4 Branches

Allgemein

Head



#1 Allgemein



Git vs. GitHub

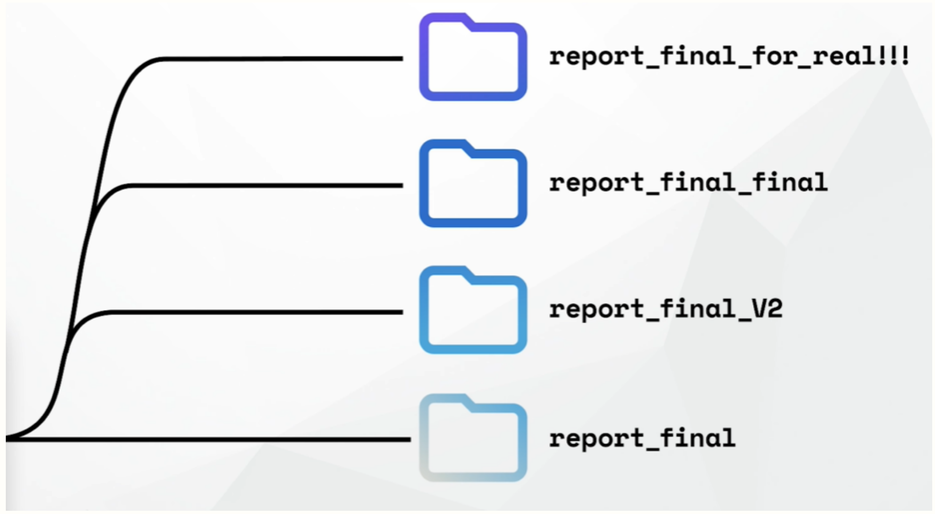
Versionskontrollsysteme

Geschichte von Git

Installation

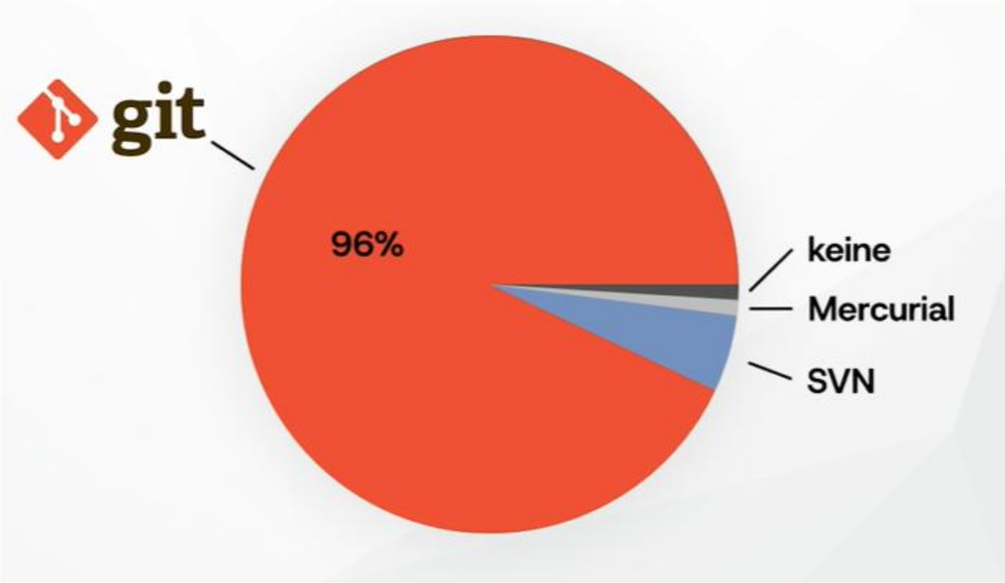
Motivation

- Angenommen Sie arbeiten mit Ihrem Team an einem Programm und zeigen dem Kunden den ersten Entwurf
  - Das Design passt dem Kunden nicht – Sie müssen es also nochmals machen
- Beim nächsten Meeting ist der Kunde noch immer nicht zufrieden – das erste Design war besser und soll nun doch verwendet werden
- Allerdings haben Sie die alten Dateien überschrieben bzw. sieht Ihre Ordnerstruktur so aus:



Motivation

- Zudem wird in der IT meistens im Team gearbeitet, und mehrere Personen wollen an einer Datei Änderungen vornehmen
  - Änderungen müssen herumgeschickt werden
- All diese Probleme löst Git seit 2005 – nämlich kostenlos
- Stack Overflow Survey 2022: Nutzung von Versionskontrolle



Git vs. GitHub

- Git ist ein lokales Werkzeug, welches wir nutzen
- GitHub ist eine Cloud-Plattform für unsere Projekte
  - Webbasierte Hostingplattform von Microsoft
  - Ermöglicht Entwicklern das speichern, verwalten und teilen von Projekten

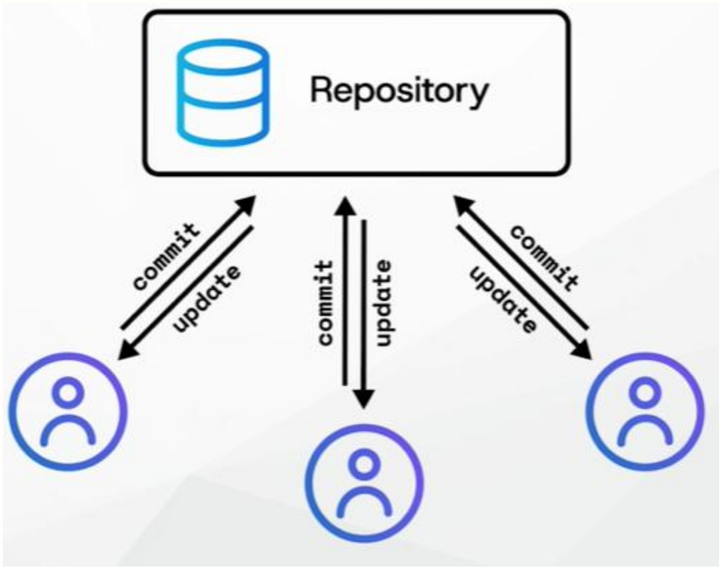
- Man könnte sagen, Git ist die Kaffeemaschine und GitHub das Kaffeehaus

Versionskontrollsysteme

- Versionskontrollsystem ist eine Software, die es erlaubt die Versionsgeschichte der Änderungen, die von verschiedenen Personen durchgeführt wurden, zu verfolgen und bei Bedarf zusammenzuführen
- Es gibt zwei Arten von Versionskontrollsystemen:
  - Zentralisierte Versionskontrolle
    - Speichert alle Änderungen an einem Projekt an einem zentralen Ort
    - Ähnlich wie bei Google Docs: Alle haben Zugriff auf die Datei und Änderungen werden direkt synchronisiert
    - Nachteil: Lokal kann nicht gearbeitet werden – eine Internetverbindung ist notwendig
  - Verteilte Versionskontrolle
    - Versionsgeschichte wird zunächst lokal auf dem PC gespeichert
    - Es gibt dadurch zwei Stufen:
      - Lokales Repository
      - Online Repository

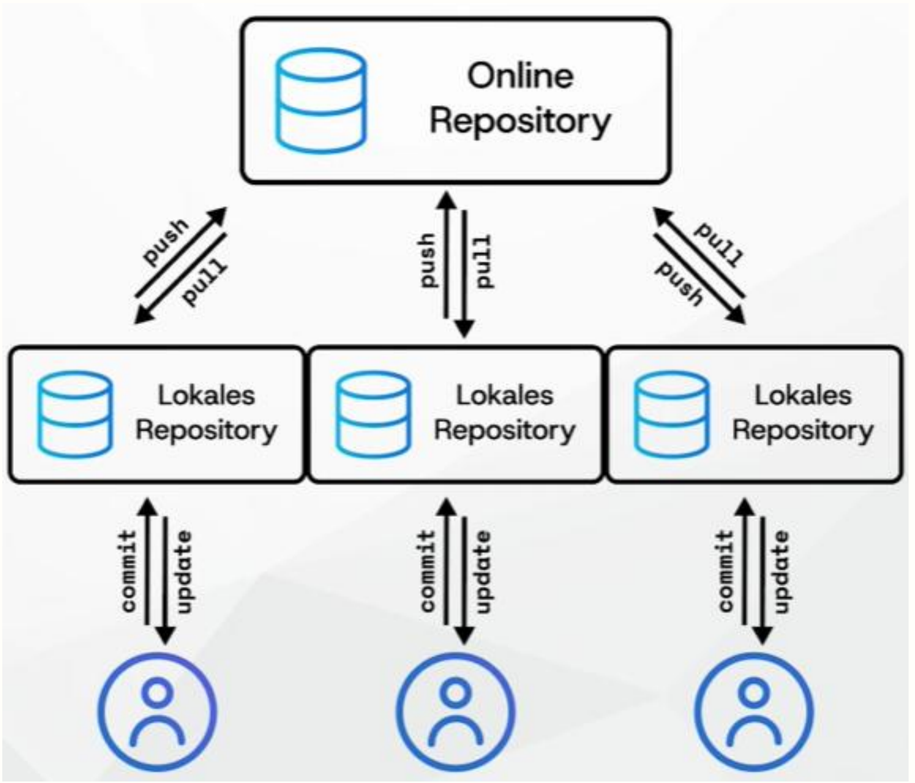
Zentralisierte Versionskontrolle

- Zentralisierte Versionskontrolle
  - Speichert alle Änderungen an einem Projekt an einem zentralen Ort
  - Ähnlich wie bei Google Docs: Alle haben Zugriff auf die Datei und Änderungen werden direkt synchronisiert
  - Nachteil: Lokal kann nicht gearbeitet werden – eine Internetverbindung ist notwendig



Verteilte Versionskontrolle

- Verteilte Versionskontrolle
  - Versionsgeschichte wird zunächst lokal auf dem PC gespeichert
  - Es gibt dadurch zwei Stufen:
    - Lokales Repository:
      - Änderungen durchführen und testen
    - Online Repository
      - Änderungen mit anderen Entwicklern zusammenführen



git commit

- Commit ist eine zentrale Funktion von git
- Friert den aktuellen Zustand des Projekts ein
- Es können mehrere commits bzw. snapshots abgespeichert werden, sodass man nachher jederzeit sehen kann, wie das Projekt in einem konkreten Moment ausgesehen hat
  - Wenn nötig, kann man zu einem gewünschten Zustand zurückgehen
- Jeder Commit bekommt von git ein Etikett aufgeklebt, in welchem vermerkt wird, wer wann was verändert hat
  - Zusätzlich bekommt jeder commit eine ID (Hash) und einen Pointer, der auf den Parent commit zeigt
  - Damit ist nachvollziehbar, in welcher Reihenfolge Änderungen durchgeführt wurden
- Tipp: Machen Sie viele kleine, statt einen großen Commit und wählen Sie eine gute Message

Geschichte von Git

- Entwickler: Linus Torvalds
  - Kennen Sie vielleicht vom Linux Kernel
- 2005 haben er und sein Team ihre Verwaltung verloren, woraufhin er sich gedacht hat, er baut selbst eines, da alles auf dem Markt zu langsam oder zu schlecht war
- Innerhalb von zwei Wochen hatte er Git fertig
- Funfact: Git = brit. Slang „Blödmann“ oder „widerlicher Typ“
- Linus sagt dazu: „Ich bin ein Egoist, also benenne ich alle meine Projekte nach mir selbst.“

Installation

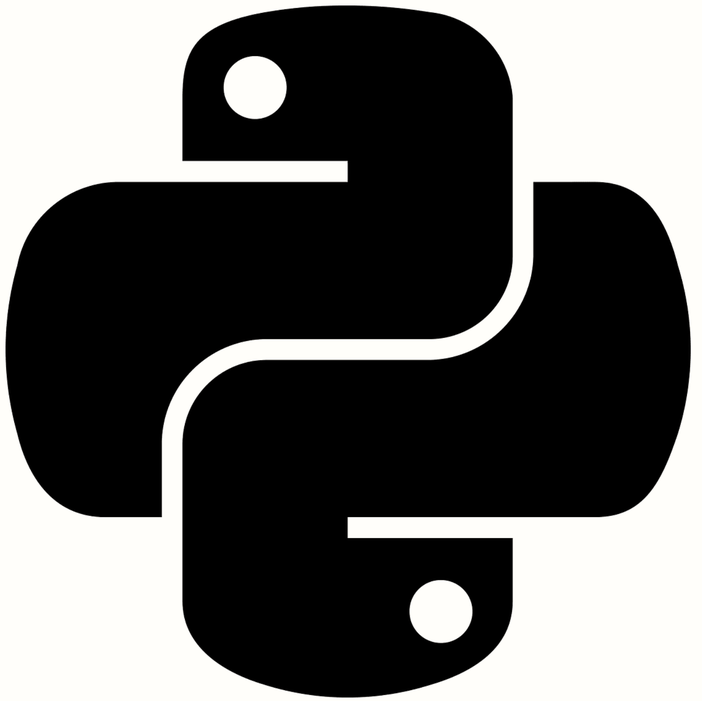
- Sollten Sie Git noch nicht installiert haben, laden Sie es hier herunter
- Sie wissen nicht, ob Git installiert ist?
  - Öffnen Sie das Terminal und geben Sie git --version ein
  - Bekommen Sie eine Versionsnummer angezeigt, ist Git bereits installiert
  - Bekommen Sie eine Fehlermeldung, ist Git nicht installiert
- Sie benötigen im weiteren Verlauf auch einen GitHub Account
  - Sollten Sie noch keinen besitzen, erstellen Sie hier einen

#2 Stages & Files

Stages

Files

Lizenzen



Die drei Stages von Git

- Es gibt drei Ebenen, in denen unser Code lebt
- Working Directory
  - Lokale Kopie unserer Dateien auf unserer Festplatte
  - Mit git add gelangen wir in die nächste Ebene
- Staging Area
  - Wie eine Bühne, d.h. die lokalen Dateien werden auf die Bühne bewegt, und erst wenn die gesamte Szene fertig ist, können wir commiten
  - Mit git commit erzeugt Git ein snapshot mit allen unseren Änderungen, die wir in die Staging Area hinzugefügt haben
  - Mit git push gelangen wir in die nächste Ebene
- Repository
  - Wir synchronisieren unsere Dateien mit der Datenbank von GitHub

Git Files

- Wenn man mit Git arbeitet, passieren viele Dinge im Hintergrund
- Damit Git zB weiß wer wir sind, was wir abspeichern wollen, was commited werden soll uvm., nutzt Git ein paar spezielle Dateien, nämlich:
  - .git
  - .gitignore
  - .gitconfig
  - README.MD

.git

- „Gehirn“ des Repositories
- Clonen wir ein Repo von GitHub bzw. initialisieren wir es lokal, erzeugt Git automatisch einen .git-Ordner
  - Ist ein versteckter Ordner, kann aber angezeigt werden
- Weist uns daraufhin, dass es sich hier um ein Git Repo handelt
- Es sollte niemals etwas manuell in diesem Ordner verändert werden
  - Git macht das automatisch, wir kommunizieren mit Git entweder über die IDE oder über die Konsole

.gitignore

- Kann jederzeit in der Hauptebene des Projekts erstellt werden
- Ist eine .txt-Datei
- Legt fest, welche Dateien ignoriert und nicht auf GitHub gepusht werden sollen
  - Beispielsweise Passwörter, große Datensätze, riesige CSV-Dateien, temporäre Dateien
- PyCharm erstellt einen .idea-Ordner, welcher nur lokal relevant ist
  - Enthält Konfigurationen für die IDE usw.
  - Sollte nicht auf GitHub gepusht werden

.gitconfig

- Unser „Ausweis“
- Sieht man nicht im Projektordner, sondern irgendwo lokal auf dem PC
- Wir haben uns in PyCharm angemeldet, Git weiß also wer wir sind
- Man kann die .gitconfig auch auf Projektebene definieren
  - Angenommen Sie arbeiten an zwei Projekten, eines von der Firma, das andere privat
  - Sie haben zB zwei verschiedene Emails bzw. Konten, dann müssen Sie sich nicht jedes Mal neu ausweisen

README.MD

- „Startseite des Projektes“ – sollte also jedes Projekt haben
- Öffnet jemand das Projekt auf GitHub, sollte aus der README.MD herausgehen, wie der Code verwendet werden kann, wie man ihn installiert, wer ihn geschrieben hat, etc.
- „Projekt ohne README ist wie ein Buch ohne Titel“
- Auf GitHub kann man ganz einfach eine README-Datei erstellen
- Es handelt sich um ein Markdown-File
  - Sehen Sie sich die Syntax dazu an, ist sehr simpel
  - Auch Bilder, Videos, Links usw. können hinzugefügt werden

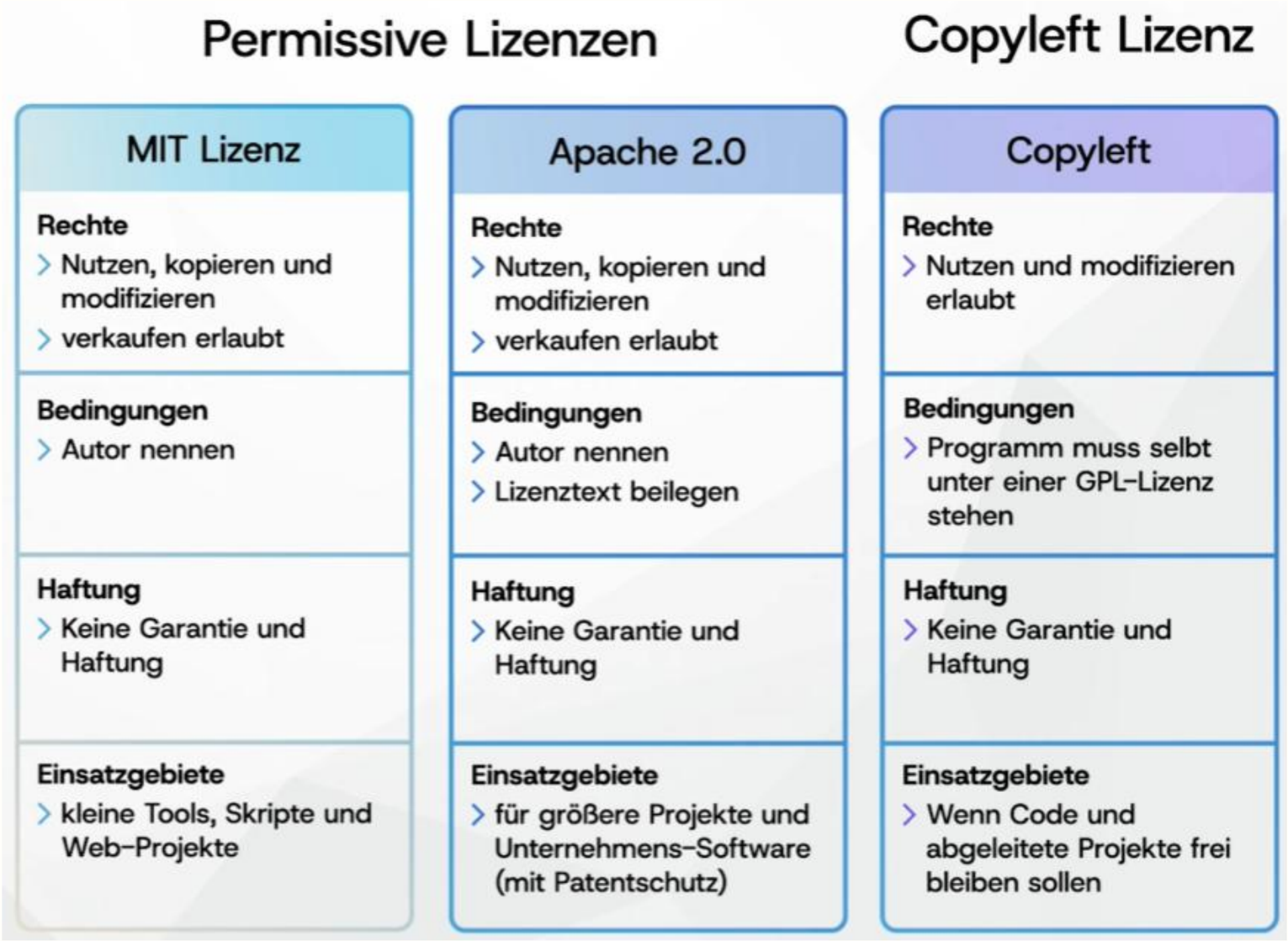
Zusammenfassung Git Files

- .git – Ordner
  - Gehirn des Projekts – Startpunkt des Repos
  - Nicht anfassen!
- .gitignore – File
  - Türsteher – lässt nur relevante Dateien hinein
  - Wird normalerweise über ein Template erzeugt
- .gitconfig – File
  - Unser Ausweis – Projektübergreifend
  - Enthält Name und E-Mail
- README.MD – File
  - Willkommensschild
  - Erklärt das Projekt

Lizenzen

- Wählt man keine Lizenz, gilt automatisch All Rights Reserved, d.h. Code darf angesehen, aber nicht verwendet, kopiert, verändert oder im eigenen Projekt eingesetzt werden
- Die Website erklärt die verschiedenen Lizenzen ausführlich
- Tipp: Schreiben Sie keine eigene Lizenz, Sie sind keine Anwälte.

Lizenzen



Permissive Lizenzen		Copyleft Lizenz
MIT Lizenz	**Apache 2.0**	**Copyleft**
Rechte › Nutzen, kopieren und modifizieren › verkaufen erlaubt	**Rechte** › Nutzen, kopieren und modifizieren › verkaufen erlaubt	**Rechte** › Nutzen und modifizieren erlaubt
Bedingungen › Autor nennen	**Bedingungen** › Autor nennen › Lizenztext beilegen	**Bedingungen** › Programm muss selbt unter einer GPL-Lizenz stehen
Haftung › Keine Garantie und Haftung	**Haftung** › Keine Garantie und Haftung	**Haftung** › Keine Garantie und Haftung
Einsatzgebiete › kleine Tools, Skripte und Web-Projekte	**Einsatzgebiete** › für größere Projekte und Unternehmens-Software (mit Patentschutz)	**Einsatzgebiete** › Wenn Code und abgeleitete Projekte frei bleiben sollen

Weitere Git-Tools

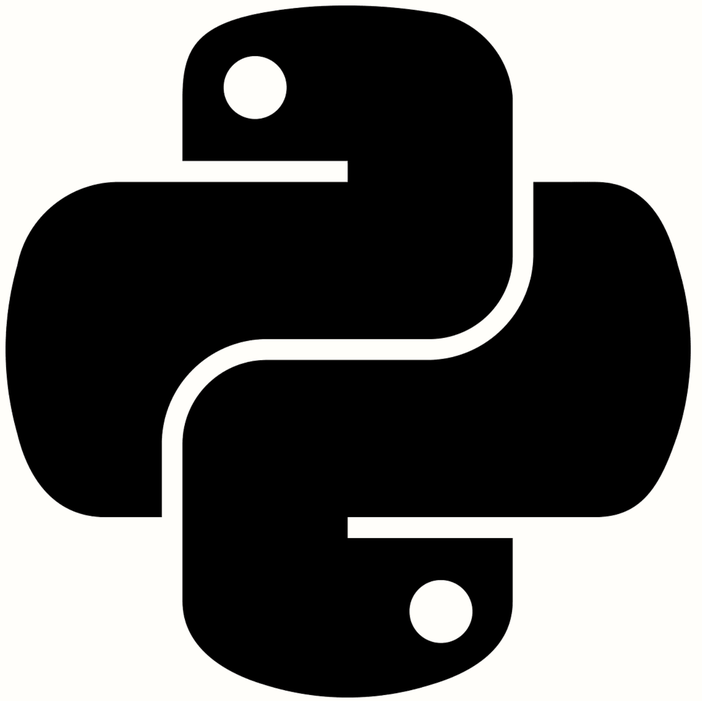
- Hier finden Sie die prominentesten Git-GUI-Tools
- Zu den beliebtesten gehören
  - GitHub Desktop – sehr ähnlich wie die Oberfläche online (kostenlos)
  - SourceTree – mittlerweile veraltetes Design (kostenlos)
  - GitKraken Desktop – modernes Design, mehrere praktische Funktionen, wird auch in der Praxis häufig verwendet (sowohl kostenlos, als auch als Lizenz)

#3 Terminal

Grundlegende Befehle

Git Befehle

HTTPS vs. SSH



Terminal

- Git kann nicht nur über GUIs oder IDEs bedient werden
- Oftmals ist das Terminal effizienter
- Hinweis: Wir nutzen die Git Bash
  - Im CMD oder der PowerShell heißen manche Befehle anders

Grundlegende Git Bash Befehle

- cd = change directory (Ordner wechseln) bzw. cd .. = ein Verzeichnis zurück
- ls = list (Alles im aktuellen Ordner anzeigen)
- mkdir = make directory (Ordner erstellen)
- touch = Datei erstellen
- cp = copy (Datei kopieren)
- mv = move (Datei verschieben)
- rm = remove (Datei löschen)
- cat = concatenate (Inhalt anzeigen)
- nano = Editor zum bearbeiten einer Datei öffnen
- start . = aktuelles Verzeichnis im Explorer öffnen

Wichtige Git Befehle

- clone = Verzeichnis kopieren
- status = Orientierung, aktuellen Zustand abfragen
- add = Dateien in die Staging Area bringen
- commit = Message eingeben und für einen push vorbereiten
- push = Änderungen hochladen
- pull = Änderungen runterladen und einbauen
- fetch = Nachschauen, ob es Änderungen gibt, ohne etwas zu verändern

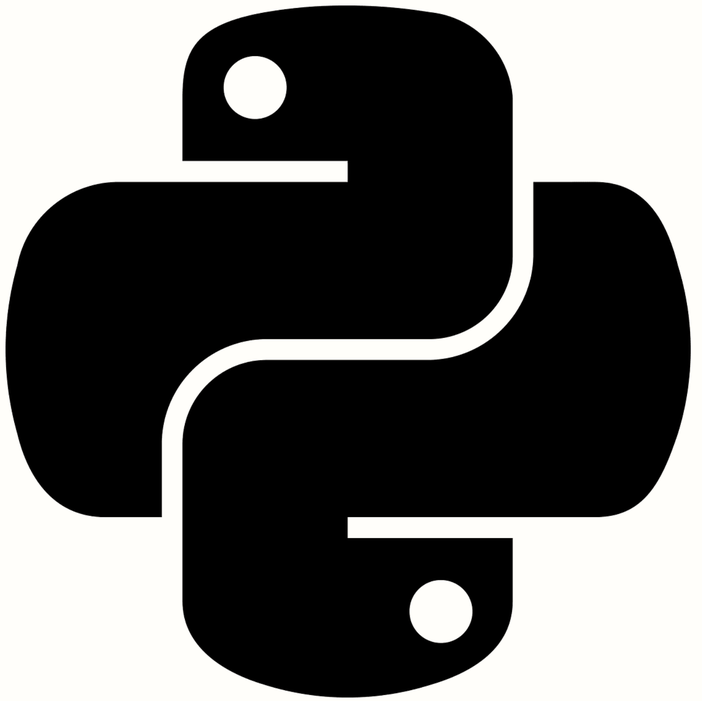
HTTPS vs. SSH

- Zum klonen eines Repos gibt es zwei verschiedene Möglichkeiten
- HTTPS
  - Vorteil: Es funktioniert sofort ohne großartige Einrichtung
  - Nachteil: Möchte man später Code hochladen, muss man sich oftmals wieder authentifizieren
- SSH
  - Bietet einen Weg uns zu authentifizieren, ohne das man sich auf der Website einloggen muss
  - Man benötigt einen lokal gespeicherten SSH-Key – sicherer als HTTPS
  - Vorteil: Einmal eingerichtet, muss man nie wieder sein Passwort eingeben
  - Nachteil: Man muss am Anfang etwas Aufwand betreiben, bis es läuft
- Für uns reicht HTTPS, hier finden Sie eine Anleitung wie Sie SSH einrichten können

#4 Branches

Allgemein

Head



Was ist ein Branch?

- Ein Branch (oder Zweig) ist eine separate Entwicklungslinie des Codes
- Wir haben bisher immer in dem main bzw. master branch gepusht
- Mit branches können wir
  - Neue Features entwickeln
  - Bugs fixen
  - Experimentieren
  - Parallel im Team arbeiten
- Dabei wird der Hauptcode nicht beeinflusst!
- Am Ende wollen wir alle branches zu einem Produkt zusammenführen

Wichtige Branch-Typen

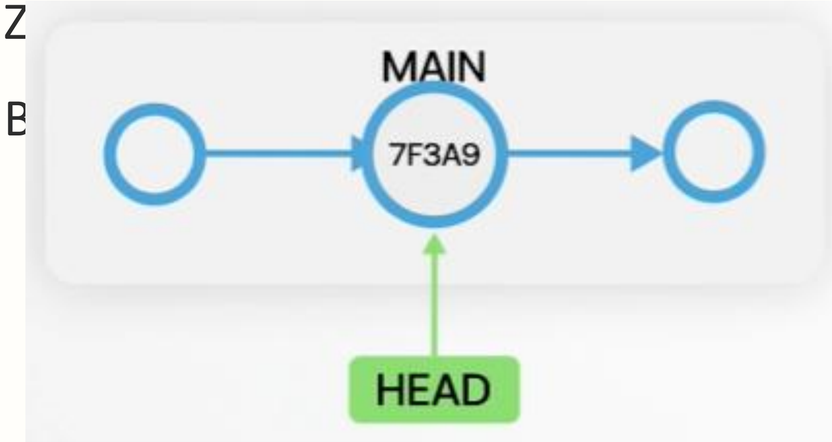
- main/master
  - Hauptbranch – wird erzeugt wenn das Repo erstellt wird
  - Enthält meistens den stabilen Produktreifen Code
- feature
  - Vom master werden feature branches erstellt, um neue features zu entwicklen und zu testen
  - Beispielsweise feature-login, feature-payments
- bugfix
  - Zur Fehlerbehbung
  - Beispielsweise bugfix-nullpointer

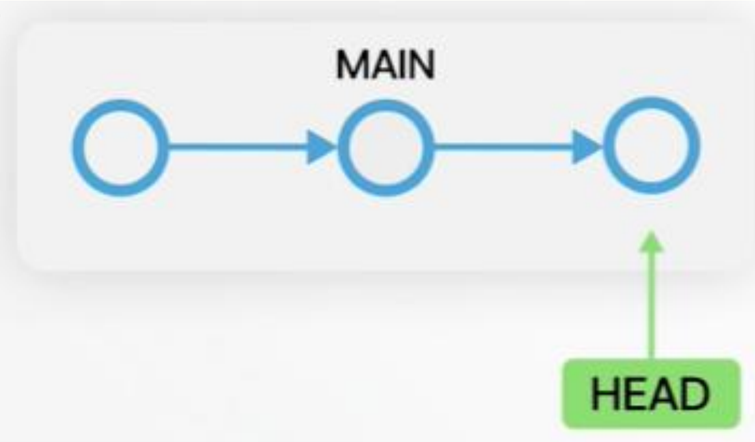
Branches intern

- Branches sind nichts anderes als Zeiger auf einen Commit
- Dadurch können branches sehr schnell erzeugt werden

Head

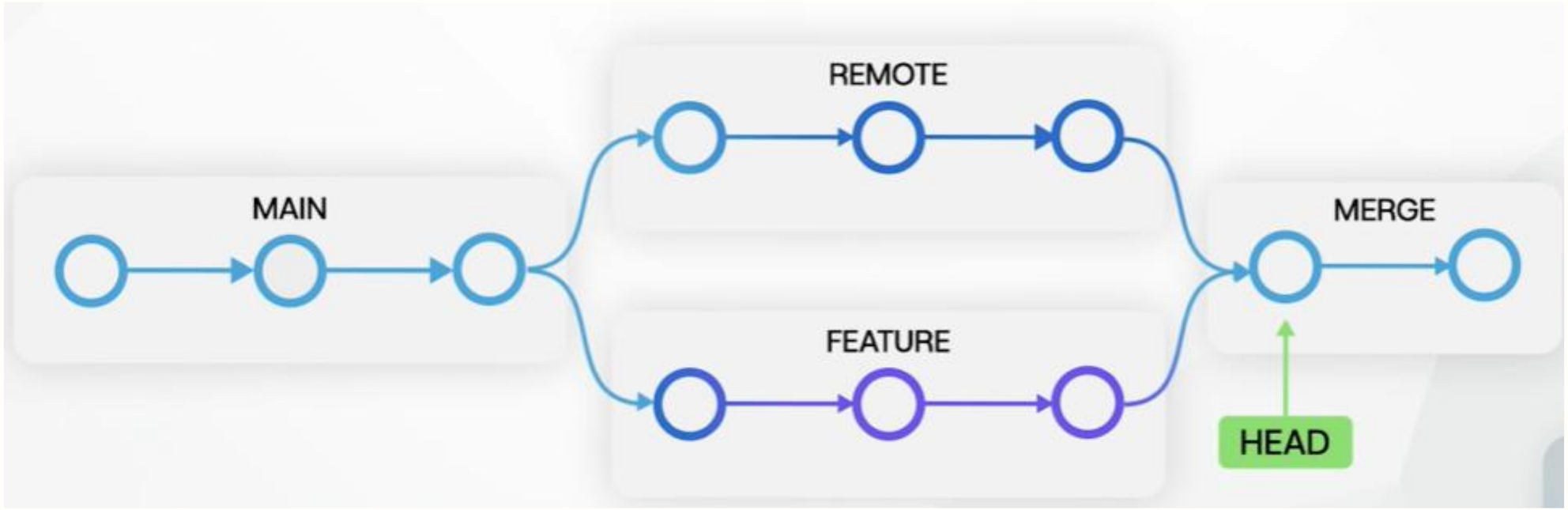
- Ist ein Pointer auf den Commit, auf dem wir uns gerade befinden, also der Commit der das aktuelle Working Directory widerspiegelt
- Führen wir einen neuen commit aus und pushen, bewegt sich auch der Head
- Angenommen wir wollen einen neuen Branch machen, dann geht der Head mit
  - Z mote, also auf Gi eren B





Head

- Nach einigen Commits und Tests sind wir sicher, dass unser Code passt, also wollen wir ihn zum main branch mergen
- Auch hier bewegt sich der Head mit



Literaturverzeichnis

- Git + GitHub Tutorial
- Pro Git, 2nd Edition – Scott Chacon